

UNIVERSIDAD NACIONAL DE PILAR
**TECNICATURA UNIVERSITARIA
EN DESARROLLO DE SOFTWARE**

WEB **DESARROLLO FULL STACK**

[2 0 2 6]

[DESARROLLO VISUAL DEL BÚNKER (CSS)]

Ya tenemos el esqueleto del búnker armado en HTML. Ahora vamos a inyectarle diseño, posicionamiento y, lo más importante, control de daños por expansión u overflow. Vamos a detallar nuestro archivo style.css línea por línea.

Cada píxel tiene una razón de ser.

Variables Globales

Líneas 1 a 8:

Línea 1: /* 1. VARIABLES GLOBALES (ESTÉTICA CTU) */

/* ... */: Esta sintaxis abre y cierra un comentario en CSS. El navegador va a ignorar por completo lo que escribamos acá adentro. Es nuestra herramienta para documentar el código y dejar mensajes para otros programadores del equipo.

Línea 2: :root {

:root: Es una pseudo-clase (un selector de estado o posición). Apunta a la raíz absoluta del documento. Lo que vamos a guardar entre estas llaves { } va a estar disponible de forma global para cualquier rincón de nuestro archivo.

Línea 3: --fondo-absoluto: #050505;

--: Los dos guiones iniciales son obligatorios. Le avisan al lenguaje CSS que estamos inventando nuestra propia variable.

fondo-absoluto: Es el nombre de fantasía que nosotros elegimos darle.

#050505: Es el valor asignado. El símbolo # indica que es un código de color Hexadecimal. Es un gris tan oscuro que el ojo lo percibe como negro, ideal para no fatigar la vista en pantallas de monitoreo.

Línea 4: --color-terminal: #00ff41; /* Fósforo Verde / Neón */

Definimos la variable para nuestro verde táctico.

Línea 5: --color-alerta: #ffb000; /* Ámbar */

Definimos la variable para el color de alerta.

Línea 6: --borde-panel: 1px solid #00ff41;

Acá guardamos una instrucción compuesta. Define 1px (un píxel de grosor), solid (una línea continua, sin cortes) y le pasamos el color verde.

Línea 7: --sombra-neon: 0 0 10px rgba(0, 255, 65, 0.2);

Variable para el brillo de los paneles.

El primer 0 indica que no hay desplazamiento de sombra en el eje X (horizontal).

El segundo 0 indica que no hay desplazamiento en el eje Y (vertical). La sombra nace desde el centro exacto de la caja.

10px es el radio de difuminado.

rgba(...): Es un modelo de color (Red, Green, Blue, Alpha). Los primeros tres números arman el color verde, y el último valor (0.2) es la opacidad (Alpha). Significa que la sombra solo tiene un 20% de intensidad. Es un brillo sutil.

Línea 8: }

Cerramos el bloque del núcleo global.

Constitución básica y reseteo

Líneas 9 a 14:

Línea 9: /* 2. LA LEY CONSTITUCIONAL Y RESETEO */

Línea 10: * {

*: Es el **Selector Universal**. Le estamos ordenando al navegador que aplique estas reglas a todas las etiquetas del HTML sin excepción.

Línea 11: box-sizing: border-box;

Obliga a que cualquier relleno (padding) o borde que le pongamos a una caja se calcule **hacia adentro** del ancho total. Garantiza que las cajas no se ensanchen misteriosamente y no sobrepasen la grilla.

Ejemplo de como cambia la matemática del navegador

Acá tenemos el ejemplo exacto de cómo cambia la matemática del navegador.

Imaginá que construís una caja con un ancho de 300px y le agregás un relleno (padding) de 50px a cada lado.

El comportamiento por defecto (content-box): El navegador suma todo hacia afuera. Ancho total real = 300px (caja) + 50px (padding izquierdo) + 50px (padding derecho) = **400px**. *Tu diseño se rompe porque la caja crece y empuja la grilla.*

(border-box): El navegador empuja el relleno hacia adentro, en vez de hacia afuera, respetando el límite estricto que le marcaste. Ancho total real = **300px**. El espacio útil para el texto adentro se achica a 200px, pero la arquitectura exterior de tu caja queda intacta.

Línea 12: margin: 0;

Resetea la distancia externa (el margen) de todos los elementos a cero.

Línea 13: padding: 0;

Resetea la distancia interna de todos los elementos a cero. Limpiamos la "basura" visual que el navegador trae de fábrica.

Línea 14: }

Tipografía y Fondo

Líneas 15 a 25:

Línea 15: /* 3. TIPOGRAFÍA Y FONDO */

Línea 16: body {

Seleccionamos la etiqueta body, el cuerpo visible de nuestra terminal.

Línea 17: background-color: var(--fondo-absoluto);

background-color: Propiedad que pinta el fondo.

var(...): Es la función que va al :root, busca la variable que le pedimos y devuelve su valor.

Línea 18: color: var(--color-terminal);

color: Propiedad que tiñe todo el texto del documento.

Línea 19: /* La estética de la terminal del MVP: fuente monoespaciada */

Línea 20: font-family: 'Courier New', Courier, monospace;

font-family: Establece la tipografía. Es una lista de prioridades. Si la compu del usuario no tiene 'Courier New', va a buscar Courier. Si tampoco la tiene, va a aplicar cualquier fuente genérica monospace. En las fuentes monoespaciadas, la letra "i" mide exactamente el mismo ancho que la letra "m". Es lo que le da el aspecto de código de programación real y vintage (es la onda del MVP – Hay que crear otra mejor).

Línea 21: height: 100vh;

height: Es la altura.

vh: Significa *Viewport Height* (Altura de la Ventana Visible). Al poner 100vh obligamos a que el cuerpo mida exactamente el 100% de la pantalla del usuario.

Línea 22: display: flex;

Encendemos el motor Flexbox. Convertimos al body en una caja flexible para mover fácil el contenido.

Línea 23: justify-content: center;

Centra el contenido en el eje principal (horizontal).

Línea 24: align-items: center;

Centra el contenido en el eje transversal (vertical). Con estas dos líneas juntas, el búnker queda en el medio exacto del monitor.

Línea 25: }

Contenedor Maestro

Líneas 26 a 34:

Línea 26: /* 4. CONTENEDOR MAESTRO */

Línea 27: .bunker-container {

El punto . indica que estamos seleccionando una class específica de nuestro HTML.

Línea 28: width: 95%;

Ocupa el 95% del ancho de la pantalla (deja un 2.5% de aire libre a cada lado).

Línea 29: max-width: 1200px;

Límite de seguridad. Si el usuario tiene un monitor gigante, el búnker no va a crecer más allá de los 1200 píxeles para no deformarse.

Línea 30: height: 90vh;

Ocupa el 90% del alto de la pantalla.

Línea 31: display: flex;

Encendemos Flexbox adentro del búnker.

Línea 32: flex-direction: column;

Por defecto, Flexbox apila las cosas en fila (horizontal). Con column, le ordenamos que apile los elementos uno **debajo** del otro (vertical).

Línea 33: gap: 20px;

Genera una trinchera o brecha de separación exacta de 20 píxeles entre la consola superior y la grilla de abajo.

Línea 34: }

Consola de Entrada

Líneas 35 a 43:

Línea 35 a 38: `.consola-input { ... }`

Activamos Flexbox en fila (predeterminado) y separamos sus hijos con un gap de 15px.

Línea 39: `padding: 20px;`

Metemos 20 píxeles de aire interno para que los textos no toquen los bordes físicos de la caja.

Línea 40 y 41: Llamamos a las variables para dibujar el borde verde y el brillo neón exterior.

Línea 42: `background-color: #0a0a0a;`

Un gris de un milímetro y más claro que el fondo absoluto para dar sensación de relieve.

Líneas 44 a 53:

Línea 44: `.consola-input input {`

Selector Descendente. Le decimos a CSS: "Buscá la etiqueta input, pero solo a la que esté adentro de la clase `.consola-input`".

Línea 45: `flex-grow: 1; /* El input absorbe todo el espacio sobrante */`

`flex-grow`: Es una orden de Flexbox. Le dice al campo de texto que crezca geométricamente y consume todo el espacio vacío de la fila. Esto empuja al botón hacia el extremo derecho.

Línea 46 y 47: `background: transparent; border: none;`

Eliminamos el fondo blanco y el borde 3D que el navegador le pone por defecto a las áreas de texto.

Línea 48: `border-bottom: 1px solid var(--color-terminal);`

Dibujamos únicamente una línea en el piso del campo de texto.

Línea 49 y 50: Heredamos el color verde y le decimos a la tipografía que use la misma del búnker (`inherit`).

Línea 51: `font-size: 1.2rem;`

Usamos una medida relativa (rem) para el tamaño de la letra.

Línea 52: outline: none;

Propiedad fundamental. Elimina ese recuadro de "foco" (generalmente azul) que aparece de la nada cuando el usuario hace clic para tipear.

Líneas 54 a 67 (El Botón y su Estado):

Línea 54: .consola-input button {

Línea 55 a 60: Le invertimos los colores (fondo verde, letra negra), le sacamos los bordes feos y le ponemos la fuente en negrita (font-weight: bold).

Línea 61: cursor: pointer;

Le dice al navegador que cuando el usuario pose el mouse encima, cambie la flechita normal por la mano indicadora de "clic".

Línea 62: transition: 0.2s;

Le ordenamos que cualquier cambio visual futuro no sea brusco, sino que suceda como una animación suave que dure 0.2 segundos.

Línea 64: .consola-input button:hover {

:hover es una pseudo-clase. Representa el estado interactivo de "mouse por encima".

Línea 65 y 66: Cuando pasamos el mouse, el botón cambia a color ámbar (--color-alerta) y emite un brillo de advertencia.

Grilla Táctica (CSS Grid)

Líneas 68 a 77:

Línea 68: /* 6. GRILLA TÁCTICA (CSS GRID) */

Línea 69: .grid-paneles {

Línea 70: display: grid;

Apagamos Flexbox y encendemos el motor bidimensional. Ahora vamos a manejar filas y columnas simultáneamente como en un tablero de ajedrez.

Línea 72: grid-template-columns: 1fr 1fr;

fr significa fracción. Acá estamos construyendo dos columnas. Le pedimos que la columna 1 ocupe 1fr y la columna 2 ocupe 1fr. Resultado: se reparten la pantalla 50% y 50% de manera matemáticamente perfecta.

Línea 74: grid-template-rows: 1fr 1fr;

Construimos dos filas exactamente iguales. Acabamos de crear nuestros cuatro cuadrantes.

Línea 76: flex-grow: 1;

Le ordena a la grilla completa que se expanda para ocupar todo el alto que quedó libre debajo de la consola superior.

Diseño de Cada Panel

Líneas 78 a 93:

Línea 79: `.panel {`

Línea 80 a 85: Reutilizamos el fondo oscuro, el borde y el brillo. Volvemos a activar Flexbox en modo *column* para que el título del panel quede arriba y los datos abajo.

Línea 87: `.panel-titulo {`

Línea 89: `border-bottom: 1px dashed var(--color-terminal);`

En vez de `solid` (línea continua), usamos `dashed` para crear una línea de trazos punteados.

Línea 90 y 91: Usamos `padding` para separar el texto de su propia línea punteada, y `margin` para alejar la línea del contenido que venga abajo.

El Control del Overflow

Líneas 94 a 104:

Línea 95: `.contenido-panel {`

Línea 98: `line-height: 1.5;`

Aumenta la altura de cada renglón a 1.5 para que la lectura de datos técnicos no sea tan apretada.

Línea 100: `.scroll-activo {`

Prestemos mucha atención acá. Esta clase es nuestro escudo protector para el Panel 3. (Inferior Izquierdo)

Línea 102: `overflow-y: auto;`

`overflow-y` controla el comportamiento vertical cuando los datos superan el tamaño físico de la caja. El valor `auto` le ordena al navegador: "Si el robot nos inyecta 500 enlaces, no rompas el diseño hacia abajo. Truncá lo que sobra y habilitá una barra de desplazamiento interna". Si nos olvidamos de esto, el diseño colapsa.

Línea 103: `max-height: 100%;`

La caja de contenido no puede superar el límite físico de su panel padre.

Líneas 105 a 111 (Embelllecimiento del Scroll):

Línea 106: `::-webkit-scrollbar {`

`::-webkit` es un pseudo-elemento que nos permite meternos en las partes privadas de la interfaz del navegador (funciona en Chrome, Edge y Opera). Acá apuntamos a la barra de desplazamiento y le forzamos un ancho de 8 píxeles (`width: 8px`).

Línea 109: `::-webkit-scrollbar-thumb {`

Apuntamos al "pulgar" o "miniatura" de la barra (el cuadradito que arrastramos con el mouse). Lo pintamos de verde táctico para mantener la inmersión en la consola.